คู่มือติดตั้งระบบออนไลน์สำหรับโรงเรียน

แอปสมุดคะแนนนักเรียน (Android) · Firebase ของโรงเรียน (แพ็กเกจฟรี)

ฉบับวันที่ 8 พฤศจิกายน 2569 (ปรับปรุงครั้งที่ 4) · กติกาสิทธิ์รุ่น v2

> สำหรับผู้ดูแลระบบของโรงเรียน (ครูคอมพิวเตอร์ / เจ้าหน้าที่ทะเบียน) ใช้เวลาประมาณ 30-45 นาที
>
> ทำบนคอมพิวเตอร์ที่เปิดเว็บเบราว์เซอร์ได้ + มือถือ Android ที่ติดตั้งแอปแล้ว
>
> เอกสารนี้ไม่มีรหัสผ่านหรือค่าลับ ค่าที่แจกให้ผู้ใช้ (รหัสเชื่อมต่อ) ไม่ใช่รหัสผ่าน

สารบัญ

1. ภาพรวม **3**

ขั้นตอนทั้งหมด 3

2. ก่อนเริ่ม **4**

3. ขั้นที่ 1 — สร้างโปรเจกต์ Firebase **5**

4. ขั้นที่ 2 — ลงทะเบียนแอป Android **6**

5. ขั้นที่ 3 — สร้างฐานข้อมูล Firestore **7**

6. ขั้นที่ 4 — เปิดการเข้าสู่ระบบ **8**

6.1 อีเมลและรหัสผ่าน (จำเป็น) 8

6.2 ภาษาของอีเมลยืนยัน (แนะนำ) 8

6.3 เข้าสู่ระบบด้วยบัญชี Google (ไม่บังคับ) 8

7. ขั้นที่ 5 — วางกติกาสิทธิ์ (สำคัญที่สุด) **9**

8. ขั้นที่ 6 — เชื่อมแอปกับโรงเรียน และสร้างโรงเรียน **10**

9. ขั้นที่ 7 — ข้อมูลพื้นฐาน เชิญผู้ใช้ และแจกรหัสเชื่อมต่อ **11**

9.1 บันทึกข้อมูลพื้นฐาน (เมนู "บันทึกข้อมูล") 11

9.2 เชิญผู้ใช้ (เมนู "สมาชิกและคำเชิญ") 11

9.3 แจกรหัสเชื่อมต่อ 12

9.4 ขั้นตอนของครู / ผู้ปกครอง (แจกเป็นเอกสารหน้าเดียวได้) 12

10. ขั้นที่ 8 — ตรวจสอบหลังติดตั้ง **13**

11. การดูแลประจำ **14**

โควตาและพื้นที่ 14

เก็บถาวรปีการศึกษาเก่า 14

ใช้งานตอนไม่มีอินเทอร์เน็ต 14

สมาชิก 14

ความเป็นส่วนตัว (PDPA) 15

12. แก้ปัญหาที่พบบ่อย **16**

ภาคผนวก ก — ไฟล์กติกาสิทธิ์ (firestore.rules รุ่น v2) **17**

ภาคผนวก ข — สำหรับผู้พัฒนาแอป: เปิดปุ่มเข้าสู่ระบบด้วย Google **22**

ภาคผนวก ค — สำหรับผู้พัฒนา: ทดสอบกติกาสิทธิ์ **23**

1. ภาพรวม

แอปสมุดคะแนนนักเรียนใช้ได้สองแบบ

แบบ	เหมาะกับ	ข้อมูลอยู่ที่
ใช้ในเครื่อง (ส่วนตัว)	ครูคนเดียวเก็บคะแนนของตัวเอง	มือถือเครื่องนั้น
เชื่อมต่อโรงเรียน (ออนไลน์)	ทั้งโรงเรียน ครูและผู้ปกครองใช้มือถือของตัวเอง	Firebase ของโรงเรียน

คู่มือนี้คือการตั้ง **แบบออนไลน์** ซึ่งแต่ละโรงเรียนมี "โปรเจกต์ Firebase" (ฐานข้อมูลบนอินเทอร์เน็ตของ Google) เป็นของตัวเอง

- **ฟรี** ใช้แพ็กเกจ Spark ไม่ต้องผูกบัตรเครดิต จึงไม่มีทางถูกเก็บเงิน
- ข้อมูลของโรงเรียนแยกจากโรงเรียนอื่นโดยสิ้นเชิง และโรงเรียนเป็นเจ้าของข้อมูลเอง
- โควตาฟรีต่อโรงเรียน: เก็บข้อมูล 1 GB, อ่าน 50,000 ครั้ง/วัน, เขียน 20,000 ครั้ง/วัน (โรงเรียนขนาดกลางใช้ได้สบาย)
- แต่ละคนเห็นข้อมูลตามบทบาท โดยกติกาสิทธิ์ที่ server ไม่ใช่แค่ซ่อนเมนูในแอป

บทบาท	เห็น/ทำได้
ผู้ดูแลระบบ	ทุกอย่าง + เชิญผู้ใช้ + ตั้งเกณฑ์เกรด
ครู	บันทึกคะแนนและรายงานเฉพาะวิชาที่สอน ครูที่ปรึกษาดูผลทุกวิชาของห้องตน
ผู้ปกครอง	ผลการเรียนของบุตรหลานที่ผูกไว้เท่านั้น

นักเรียนไม่ใช้แอปนี้ (แอปประกาศกลุ่มอายุ 18 ปีขึ้นไปใน Google Play)

ขั้นตอนทั้งหมด

ขั้น	ทำที่	งาน
1	คอมพิวเตอร์	สร้างโปรเจกต์ Firebase
2	คอมพิวเตอร์	ลงทะเบียนแอป Android และดาวน์โหลด `google-services.json`
3	คอมพิวเตอร์	สร้างฐานข้อมูล Firestore
4	คอมพิวเตอร์	เปิดการเข้าสู่ระบบด้วยอีเมล (และ Google ถ้าต้องการ)
5	คอมพิวเตอร์	วางกติกาสิทธิ์ `firestore.rules`
6	มือถือ	เชื่อมแอปกับโรงเรียน สมัคร และสร้างโรงเรียน
7	มือถือ	บันทึกข้อมูลพื้นฐาน เชิญผู้ใช้ แจกรหัสเชื่อมต่อ
8	ทั้งสองที่	ตรวจสอบหลังติดตั้ง

> ชื่อเมนูใน Firebase console เป็นภาษาอังกฤษตามที่ Google แสดง ถ้าบัญชี Google ตั้งภาษาไทย ชื่อเมนูจะเป็นภาษาไทยแทน แต่ตำแหน่งเหมือนกัน

2. ก่อนเริ่ม

- **บัญชี Google ของโรงเรียน** ที่จะเป็นเจ้าของโปรเจกต์ ควรเป็นบัญชีกลางของโรงเรียน (เช่น บัญชี Google Workspace for Education) ไม่ใช่บัญชีส่วนตัวของครูที่อาจย้ายโรงเรียน
- เปิด **การยืนยันตัวตนแบบ 2 ขั้นตอน** ให้บัญชีนั้น เพราะใครได้บัญชีนี้ไปจะเข้าถึงข้อมูลคะแนนทั้งหมดได้
- **อีเมลของผู้ดูแลระบบในแอป** (ใช้ได้ทั้งบัญชีเดียวกันหรือคนละบัญชี)
- มือถือ Android ที่ติดตั้งแอป "สมุดคะแนนนักเรียน" แล้ว

3. ขั้นที่ 1 — สร้างโปรเจกต์ Firebase

1. เปิด https://console.firebase.google.com แล้วเข้าสู่ระบบด้วยบัญชี Google ของโรงเรียน
2. กด **Create a project** (หรือ Add project)
3. ตั้งชื่อโปรเจกต์ เช่น `scorebook-โรงเรียนของคุณ` (ระบบจะสร้างรหัสโปรเจกต์ภาษาอังกฤษให้เอง เช่น `scorebook-abc-1a2b3`)
4. ยอมรับข้อตกลง กด Continue
5. หน้า Google Analytics ให้ **ปิด** (ไม่จำเป็นต่อแอปนี้) แล้วกด Create project
6. รอจนขึ้น "Your new project is ready" กด Continue

> ตรวจที่มุมล่างซ้ายของ console ต้องเห็นคำว่า **Spark** (แพ็กเกจฟรี) ห้ามกด Upgrade ถ้าไม่ต้องการจ่ายเงิน

4. ขั้นที่ 2 — ลงทะเบียนแอป Android

1. ที่หน้าแรกของโปรเจกต์ กดไอคอน **Android** (หรือ Project settings ⚙ ⟶ General ⟶ Your apps ⟶ Add app ⟶ Android)
2. ช่อง **Android package name** ใส่ให้ตรงทุกตัวอักษร: `com.studentscore.app`
3. ช่อง App nickname ใส่อะไรก็ได้ เช่น `สมุดคะแนน`
4. ช่อง **Debug signing certificate SHA-1** เว้นว่างได้ (ไม่ต้องใช้กับแอปนี้)
5. กด **Register app**
6. กด **Download google-services.json** เก็บไฟล์ไว้ ไฟล์นี้ใช้ในขั้นที่ 6
7. ขั้นที่เหลือ (Add Firebase SDK) กด Next ไปจนจบ แล้ว Continue to console — **ไม่ต้องทำอะไร** เพราะแอปตั้งค่าเองจากไฟล์ในข้อ 6

> ส่งไฟล์ `google-services.json` เข้ามือถือของผู้ดูแลระบบ เช่น อีเมลถึงตัวเอง, LINE (Keep), หรือ Google Drive แล้วดาวน์โหลดลงมือถือ

5. ขั้นที่ 3 — สร้างฐานข้อมูล Firestore

1. เมนูซ้าย **Build** ⟶ **Firestore Database** ⟶ กด **Create database**
2. ถ้ามีให้เลือก Edition ให้เลือก **Standard** แล้ว **Location** เลือกภูมิภาคเอเชียที่ใกล้ที่สุด เช่น `asia-southeast1 (Singapore)` — **เลือกแล้วเปลี่ยนภายหลังไม่ได้**
3. เลือก **Start in production mode** (ห้ามเลือก test mode เพราะ test mode เปิดให้ทุกคนอ่านคะแนนได้)
4. กด Create

> ถ้าเผลอเลือก test mode ไม่เป็นไร ขั้นที่ 5 จะทับกติกาให้ปลอดภัย และแอปจะไม่ยอมทำงานกับฐานข้อมูลที่เปิดโล่ง

6. ขั้นที่ 4 — เปิดการเข้าสู่ระบบ

6.1 อีเมลและรหัสผ่าน (จำเป็น)

1. เมนู **Build** ⟶ **Authentication** ⟶ กด **Get started**
2. แท็บ **Sign-in method** ⟶ เลือก **Email/Password**
3. เปิดสวิตช์ **Email/Password** อันแรก (ไม่ต้องเปิด Email link) ⟶ **Save**

6.2 ภาษาของอีเมลยืนยัน (แนะนำ)

1. Authentication ⟶ แท็บ **Templates**
2. กดไอคอนดินสอ (Template language) เลือก **Thai** ⟶ Save
3. ผู้ใช้จะได้รับอีเมลยืนยันและอีเมลตั้งรหัสใหม่เป็นภาษาไทย (ผู้ส่งคือ `noreply@<รหัสโปรเจกต์>.firebaseapp.com`)

6.3 เข้าสู่ระบบด้วยบัญชี Google (ไม่บังคับ)

ปุ่ม "เข้าสู่ระบบด้วยบัญชี Google" ในแอปจะทำงานเมื่อผู้พัฒนาแอปเปิดใช้แล้ว
และโรงเรียนอนุญาตรหัสของแอปไว้ ถ้าในแอปไม่มีปุ่มนี้ ให้ข้ามข้อนี้ (ทุกคนใช้อีเมล+รหัสได้)

1. Authentication ⟶ Sign-in method ⟶ **Add new provider** ⟶ **Google** ⟶ เปิดสวิตช์
2. **Project support email** เลือกอีเมลของโรงเรียน
3. เปิดหัวข้อ **Safelist client IDs from external projects** แล้ววาง **Web client ID ของแอป** ที่ผู้พัฒนาแจ้ง (ลงท้ายด้วย `.apps.googleusercontent.com`) กด Add
4. Save

> ทำไมต้องวางรหัสนี้: Google อนุญาตให้แอปหนึ่งตัวลงทะเบียนการเข้าสู่ระบบด้วย Google ได้ในโปรเจกต์เดียว แอปจึงลงทะเบียนไว้ที่โปรเจกต์กลางของผู้พัฒนา แล้วให้แต่ละโรงเรียน "อนุญาต" รหัสนั้น

7. ขั้นที่ 5 — วางกติกาสิทธิ์ (สำคัญที่สุด)

กติกาสิทธิ์คือสิ่งที่ทำให้ครูเห็นเฉพาะวิชาที่สอน และผู้ปกครองเห็นเฉพาะบุตรหลาน ต้องวางให้ครบทั้งไฟล์

วิธีได้ไฟล์กติกา (เลือกอย่างใดอย่างหนึ่ง)

- ในแอป: ขั้นที่ 6 แอปจะแจ้งว่า "โรงเรียนยังไม่ได้ติดตั้งกติกาสิทธิ์" ให้กด **ส่งไฟล์กติกาสิทธิ์ไปที่คอมพิวเตอร์** แล้วส่งทางอีเมล/LINE ถึงตัวเอง
- จากโค้ดต้นฉบับ: ไฟล์ `firebase/firestore.rules`
- จากภาคผนวก ก ท้ายเอกสารนี้

วิธีวาง

1. Firestore Database ⟶ แท็บ **Rules**
2. ลบข้อความเดิมในช่องแก้ไข **ทั้งหมด**
3. วางกติกาทั้งไฟล์ (ตั้งแต่ `rules_version = '2';` จนถึงวงเล็บปิดตัวสุดท้าย)
4. กด **Publish** (ถ้ามีตัวแดงแสดงข้อผิดพลาด แปลว่าคัดลอกไม่ครบ ให้คัดลอกใหม่)
5. รอประมาณ 1 นาทีให้กติกามีผล

> ถ้าในอนาคตผู้พัฒนาออกกติการุ่นใหม่ (เช่น v2) แอปจะแจ้งเตือน ให้วางไฟล์รุ่นใหม่ทับด้วยวิธีเดียวกัน

8. ขั้นที่ 6 — เชื่อมแอปกับโรงเรียน และสร้างโรงเรียน

ทำบนมือถือของผู้ดูแลระบบ

1. เปิดแอป ⟶ **เชื่อมต่อโรงเรียน (ออนไลน์)**
2. กด **นำเข้าไฟล์ google-services.json** แล้วเลือกไฟล์จากขั้นที่ 2
    - ถ้าแอปแจ้ง "ไฟล์นี้ยังไม่ได้ลงทะเบียนแอป Android ชื่อ com.studentscore.app" แปลว่าชื่อ package ในขั้นที่ 2 พิมพ์ผิด ให้เพิ่มแอปใหม่ด้วยชื่อที่ถูกต้องแล้วดาวน์โหลดไฟล์ใหม่
3. แอปตรวจกติกาสิทธิ์
    - ขึ้น "ยังไม่ได้ติดตั้งกติกาสิทธิ์" หรือ "กติกาสิทธิ์เปิดให้ทุกคนอ่าน/เขียนได้" ⟶ กลับไปทำขั้นที่ 5 แล้วกด **ตรวจอีกครั้ง**
4. หน้าเข้าสู่ระบบ กด **ยังไม่มีบัญชี สมัคร** ใส่อีเมลของผู้ดูแลระบบ ตั้งรหัสผ่าน (อย่างน้อย 8 ตัว)
5. เปิดอีเมล กดลิงก์ยืนยัน (ไม่พบให้ดูในจดหมายขยะ) แล้วกลับมาที่แอปกด **ยืนยันแล้ว**
6. แอปขึ้นหน้า **ตั้งค่าโรงเรียนใหม่** ใส่ชื่อโรงเรียน จังหวัด และชื่อของคุณ ⟶ **สร้างโรงเรียนและเป็นผู้ดูแลระบบ**

> **ทำข้อ 6 ทันทีหลังวางกติกา** คนแรกที่สร้างโรงเรียนในโปรเจกต์จะเป็นผู้ดูแลระบบ และทำได้ครั้งเดียว (อย่าแจกรหัสเชื่อมต่อให้ใครก่อนสร้างโรงเรียนเสร็จ)

9. ขั้นที่ 7 — ข้อมูลพื้นฐาน เชิญผู้ใช้ และแจกรหัสเชื่อมต่อ

9.1 บันทึกข้อมูลพื้นฐาน (เมนู "บันทึกข้อมูล")

ทำตามลำดับนี้ เพราะข้อมูลต้องผูกกัน

1. **ครู** — รหัสครู ชื่อ
2. **ห้องเรียน** — ชื่อห้อง ปีการศึกษา ครูที่ปรึกษา
3. **นักเรียน** — รหัส ชื่อ ห้อง
4. **รายวิชา** — รหัสวิชา ชื่อ หน่วยกิต ภาคเรียน **ครูผู้สอน** และ **ห้องเรียน**

นำเข้านักเรียนจาก Excel ทีละมาก ๆ (แนะนำ)

1. เมนูบันทึกข้อมูล ⟶ แท็บนักเรียน ⟶ **แบบฟอร์ม** บันทึกไฟล์แบบฟอร์มลงมือถือ (หรือใช้ไฟล์รายชื่อเดิมของโรงเรียนก็ได้)
2. คอลัมน์ที่รู้จัก: รหัสนักเรียน, ชื่อ, นามสกุล (หรือ ชื่อ-นามสกุล ช่องเดียว), ห้อง, ชื่อผู้ปกครอง, โทรศัพท์ผู้ปกครอง, อีเมลผู้ปกครอง — ต้องมีอย่างน้อย **รหัสนักเรียน** และ **ชื่อ**
3. กด **นำเข้า Excel/CSV** เลือกไฟล์ (.xlsx หรือ .csv ทั้ง UTF-8 และแบบที่ Excel ภาษาไทยบันทึก)
4. แอปแสดงสรุปก่อนนำเข้า: จำนวนเพิ่มใหม่/แก้ไข (จับคู่ด้วยรหัสนักเรียน), ห้องที่ยังไม่มี (ติ๊กให้สร้างอัตโนมัติ), แถวที่ข้ามพร้อมเหตุผล
5. นำเข้าซ้ำได้เพื่ออัปเดต — คอลัมน์ที่ไม่มีในไฟล์จะคงค่าเดิมไว้

> ครูจะเห็นนักเรียนในห้องของวิชาที่ตนสอนโดยอัตโนมัติ ไม่ต้องผูกครูกับนักเรียนทีละคน
>
> วิชาที่มีคะแนนแล้วเปลี่ยนห้องเรียนไม่ได้ (ให้สร้างวิชาใหม่สำหรับห้องอื่น) แต่เปลี่ยนครูผู้สอนได้

9.2 เชิญผู้ใช้ (เมนู "สมาชิกและคำเชิญ")

กด **เชิญผู้ใช้** แล้วใส่

- **อีเมล** ที่ผู้ใช้จะใช้เข้าสู่ระบบ
- **บทบาท** และการผูกข้อมูล
  - ครู ⟶ เลือกครูจากข้อมูลครู
  - ผู้ปกครอง ⟶ เลือกนักเรียนในความปกครอง (เลือกได้หลายคน พ่อกับแม่ให้เชิญคนละอีเมล)
  - ผู้ดูแลระบบ ⟶ **แนะนำให้มีอย่างน้อย 2 คน** เผื่อคนหนึ่งเข้าไม่ได้

เชิญผู้ปกครองทั้งห้องในครั้งเดียว: กด **เชิญผู้ปกครองทั้งห้อง** เลือกห้อง
แอปใช้อีเมลผู้ปกครองจากข้อมูลนักเรียน (ที่นำเข้าจาก Excel)

- พี่น้องที่ใช้อีเมลผู้ปกครองเดียวกัน ได้คำเชิญเดียวที่ผูกลูกทุกคน
- ผู้ปกครองที่เป็นสมาชิกอยู่แล้ว ระบบเพิ่มบุตรหลานให้เลย ไม่ต้องรับคำเชิญใหม่

- รายการที่ไม่มีอีเมล อีเมลผิดรูปแบบ หรือได้รับเชิญแล้ว จะแสดงเป็น "ข้าม" พร้อมเหตุผล

คำเชิญจะรออยู่ในแท็บ "คำเชิญที่รอรับ" จนกว่าผู้ใช้จะกดยอมรับ แล้วย้ายไปอยู่ในแท็บ "สมาชิก"

9.3 แจกรหัสเชื่อมต่อ

เมนู **ตั้งค่า ⟶ โรงเรียน (ออนไลน์)**

- **แสดง QR** ให้ผู้ใช้สแกน (เช่น ฉายบนจอในห้องประชุมผู้ปกครอง)
- **แชร์** ส่งรหัสทาง LINE กลุ่มครู/ผู้ปกครอง

> รหัสเชื่อมต่อไม่ใช่รหัสผ่าน ใครได้รหัสไปก็เข้าไม่ได้ถ้าไม่ได้รับเชิญ แต่ไม่ควรโพสต์ในที่สาธารณะ

9.4 ขั้นตอนของครู / ผู้ปกครอง (แจกเป็นเอกสารหน้าเดียวได้)

1. ติดตั้งแอป "สมุดคะแนนนักเรียน" จาก Google Play
2. เปิดแอป ⟶ **เชื่อมต่อโรงเรียน (ออนไลน์) ⟶ สแกน QR รหัสเชื่อมต่อ** (หรือวางรหัส)
3. **สมัคร** ด้วยอีเมลที่แจ้งโรงเรียนไว้ แล้วตั้งรหัสผ่านเอง (หรือกด "เข้าสู่ระบบด้วยบัญชี Google" ถ้ามี)
4. เปิดอีเมล กดลิงก์ยืนยัน แล้วกลับมากด **ยืนยันแล้ว**
5. กด **ยอมรับคำเชิญ** — ใช้งานได้ทันที
6. ลืมรหัสผ่าน: หน้าเข้าสู่ระบบ ใส่อีเมล แล้วกด **ลืมรหัสผ่าน** ระบบส่งลิงก์ตั้งรหัสใหม่ทางอีเมล

10. ขั้นที่ 8 — ตรวจสอบหลังติดตั้ง

- [ ] มุมล่างซ้ายของ Firebase console เป็นแพ็กเกจ **Spark**
- [ ] Firestore ⟶ Rules มีบรรทัด `allow get: if v == 'v2';` และกด Publish แล้ว
- [ ] แอปเปิดแดชบอร์ดได้ ไม่ขึ้นคำเตือนเรื่องกติกาสิทธิ์
- [ ] ทดลองเชิญบัญชีผู้ปกครองทดสอบ แล้วเข้าสู่ระบบเป็นผู้ปกครอง ต้อง **เห็นเฉพาะบุตรหลานที่ผูกไว้** ดูของนักเรียนคนอื่นไม่ได้
- [ ] ทดลองเชิญบัญชีครูทดสอบ ต้องบันทึกคะแนนได้เฉพาะวิชาที่สอน
- [ ] มีผู้ดูแลระบบในแอปอย่างน้อย 2 คน
- [ ] เพิ่มเจ้าของโปรเจกต์สำรองอีก 1 คน: Project settings ⚙ ⟶ **Users and permissions** ⟶ Add member ⟶ บทบาท **Owner** (กันกรณีบัญชีหลักใช้ไม่ได้)

11. การดูแลประจำ

โควตาและพื้นที่

- ดูในแอป: ตั้งค่า ⟶ **พื้นที่ข้อมูลออนไลน์** (ค่าประมาณ)
- ดูค่าจริง: Firebase console ⟶ Firestore Database ⟶ แท็บ **Usage**
- ถ้าใช้โควตารายวันหมด ระบบจะใช้ไม่ได้ชั่วคราวจนโควตารีเซ็ต (ประมาณ 14:00-15:00 น. เวลาไทย) ข้อมูลที่ครูกรอกระหว่างนั้นจะค้างในมือถือ แล้วส่งขึ้นเองหลังรีเซ็ต
- พื้นที่ 1 GB น่าจะพอใช้ 10 ปีขึ้นไปสำหรับโรงเรียนขนาดกลาง ถ้าใกล้เต็ม (แอปเตือนที่ 80%) ให้เก็บถาวรปีการศึกษาเก่า

เก็บถาวรปีการศึกษาเก่า

ตั้งค่า ⟶ **ปีการศึกษา (เก็บถาวร/ลบ)**

1. กด **ส่งออก** ของปีนั้น ได้ไฟล์ Excel: ชีต "สรุป" (นักเรียน × วิชา คะแนนรวม เกรด) + ชีตละรายวิชาพร้อมคะแนนทุกรายการ — เก็บไฟล์ไว้ในที่ปลอดภัย
2. ปุ่ม **ลบ** ใช้ได้หลังส่งออกแล้วเท่านั้น ต้องพิมพ์ปี พ.ศ. ยืนยัน
3. ลบรายวิชา คะแนน และประวัติแก้คะแนนของปีนั้น นักเรียนไม่ถูกลบ ห้องเรียนของปีนั้นลบเฉพาะห้องที่ไม่มีนักเรียนแล้ว

ใช้งานตอนไม่มีอินเทอร์เน็ต

เครื่องที่เคยเข้าใช้ออนไลน์แล้ว เปิดแอปตอนไม่มีเน็ตได้ จะเห็นข้อมูลที่เคยโหลดไว้ และบันทึกคะแนนได้ ข้อมูลจะค้างในเครื่องแล้วส่งขึ้นเองเมื่อกลับมาออนไลน์ (อย่าถอนการติดตั้งแอประหว่างนั้น)

สมาชิก

- ครูย้ายออก / ผู้ปกครองของนักเรียนที่จบแล้ว: เมนูสมาชิกและคำเชิญ ⟶ แตะชื่อ ⟶ ปิด **ใช้งานได้** (หรือลบ) สิทธิ์หมดทันที
- ผู้ใช้ลบบัญชีของตนเองได้ที่ ตั้งค่า ⟶ บัญชีผู้ใช้ ⟶ ลบบัญชีของฉัน (คะแนนที่โรงเรียนบันทึกไม่ถูกลบ) — ถ้าผู้ดูแลระบบคนสุดท้ายลบบัญชี เจ้าของโปรเจกต์ต้องเพิ่มผู้ดูแลใหม่ใน Firebase console (Firestore ⟶ collection `members` ⟶ เอกสาร uid ของผู้ใช้ ใส่ role = ADMIN) จึงควรมีผู้ดูแลอย่างน้อย 2 คนเสมอ
- ประวัติการแก้คะแนนทุกครั้งเก็บถาวร (ใคร เมื่อไร เหตุผล) ดูได้ที่ ตั้งค่า ⟶ ประวัติการแก้ไขคะแนน แก้ไขหรือลบประวัติไม่ได้

ความเป็นส่วนตัว (PDPA)

คะแนนนักเรียนเป็นข้อมูลส่วนบุคคล โรงเรียนเป็นผู้ควบคุมข้อมูล ควรแจ้งผู้ปกครองว่าเก็บข้อมูลอะไร ไว้ที่ใด (Google Firebase ภูมิภาคที่เลือกในขั้นที่ 3) และใครเข้าถึงได้

12. แก้ปัญหาที่พบบ่อย

อาการในแอป	สาเหตุ / วิธีแก้
ไฟล์นี้ยังไม่ได้ลงทะเบียนแอป Android ชื่อ com.studentscore.app	ชื่อ package ในขั้นที่ 2 ไม่ตรง เพิ่มแอปใหม่ให้ถูกต้อง แล้วดาวน์โหลดไฟล์ใหม่
โรงเรียนยังไม่ได้ติดตั้งกติกาสิทธิ์	ยังไม่ได้ทำขั้นที่ 5 หรือวางไม่ครบ หรือยังไม่กด Publish
กติกาสิทธิ์เปิดให้ทุกคนอ่าน/เขียนได้	ฐานข้อมูลเป็น test mode ให้ทำขั้นที่ 5
โรงเรียนยังไม่ได้เปิดวิธีเข้าสู่ระบบนี้ใน Firebase	ยังไม่ได้เปิด Email/Password ในขั้นที่ 4
ไม่ได้รับอีเมลยืนยัน / ตั้งรหัสใหม่	ดูจดหมายขยะ, กด "ส่งอีเมลยืนยันอีกครั้ง", ตรวจว่าพิมพ์อีเมลถูก
ยังไม่ได้รับเชิญ	อีเมลที่สมัครไม่ตรงกับคำเชิญ ให้ผู้ดูแลตรวจอีเมลในแท็บคำเชิญ (ตัวพิมพ์เล็ก/ใหญ่ไม่มีผล)
ระบบออนไลน์ใช้โควตาฟรีของวันนี้ครบแล้ว	รอโควตารีเซ็ตประมาณบ่าย 2-3 โมง ข้อมูลที่กรอกไว้จะส่งขึ้นเอง
ไม่มีปุ่ม "เข้าสู่ระบบด้วยบัญชี Google"	ผู้พัฒนายังไม่ได้เปิดใช้ ใช้อีเมล+รหัสแทน
เข้าด้วย Google แล้วผิดพลาด	ขั้นที่ 6.3 ยังไม่ได้ safelist Web client ID ของแอป
เชื่อมต่อ server ไม่ได้	ตรวจอินเทอร์เน็ต แล้วกด "ลองใหม่" (เครื่องที่ยังไม่เคยเข้าใช้ต้องมีเน็ตครั้งแรก)
นำเข้าแล้วขึ้น "ไม่พบหัวตาราง"	แถวหัวตารางต้องมีคอลัมน์ รหัสนักเรียน และ ชื่อ อยู่ใน 10 แถวแรก ดูตัวอย่างจากปุ่ม "แบบฟอร์ม"
เชิญผู้ปกครองทั้งห้องแล้วข้ามหลายคน	ข้อมูลนักเรียนยังไม่มีอีเมลผู้ปกครอง ให้เพิ่มคอลัมน์ อีเมลผู้ปกครอง ในไฟล์แล้วนำเข้าซ้ำ

ภาคผนวก ก — ไฟล์กติกาสิทธิ์ (firestore.rules รุ่น v2)

คัดลอกทั้งหมดในกรอบด้านล่างไปวางในขั้นที่ 5 (ถ้าคัดลอกจาก Word แล้ว Publish ไม่ผ่าน ให้ใช้ปุ่ม "ส่งไฟล์กติกาสิทธิ์" ในแอปแทน)

```
rules_version = '2';


// กติกาสิทธิ์ของแอปสมุดคะแนนนักเรียน (หนึ่งโปรเจกต์ Firebase = หนึ่งโรงเรียน)
// ผู้ดูแลระบบของโรงเรียนคัดลอกไฟล์นี้ไปวางที่ Firebase console → Firestore → Rules แล้วกด Publish
// รุ่น v2 (8 พ.ย. 2569): ผู้ใช้ลบบัญชีสมาชิกของตนเองได้
// แก้ไฟล์นี้แล้วให้เพิ่มเลข RULES_VERSION ทั้งที่นี่ (match /rulesVersion) และใน
CloudStore.RULES_VERSION ของแอป
//
// บทบาท: ADMIN ผู้ดูแลระบบ, TEACHER ครู, STUDENT นักเรียน, PARENT ผู้ปกครอง
// - ครูเขียนคะแนนได้เฉพาะวิชาที่ subjects.teacherId ตรงกับครูที่ผูกกับบัญชี
// - ครูอ่านคะแนนวิชาที่สอน และทุกวิชาของห้องที่เป็นที่ปรึกษา
// - นักเรียน/ผู้ปกครองอ่านได้เฉพาะคะแนนของตน/บุตรหลาน
// - ทุกบัญชีต้องยืนยันอีเมลแล้ว และสถานะสมาชิกต้อง active


service cloud.firestore {
  match /databases/{database}/documents {


    function dataAt(path) { return get(path).data; }
    function signedIn() { return request.auth != null; }
    function verified() { return signedIn() && request.auth.token.email_verified
== true; }
    function myEmail() { return request.auth.token.email.lower(); }
    function memberPath() { return
/databases/$(database)/documents/members/$(request.auth.uid); }
    function me() { return get(memberPath()).data; }
    function isMember() { return verified() && exists(memberPath()) && me().active
== true; }
    function isAdmin() { return isMember() && me().role == 'ADMIN'; }
    function isTeacher() { return isMember() && me().role == 'TEACHER'; }
    function isStaff() { return isMember() && me().role in ['ADMIN', 'TEACHER']; }
    function myTeacherId() { return me().teacherId; }
    function subject(id) { return
dataAt(/databases/$(database)/documents/subjects/$(id)); }
    function classroom(id) { return
dataAt(/databases/$(database)/documents/classrooms/$(id)); }
    function teaches(subjectId) {
      return isTeacher() && myTeacherId() != null && subject(subjectId).teacherId
== myTeacherId();
    }
    function canEditSubject(subjectId) { return isAdmin() || teaches(subjectId); }
    function isStr(v, maxLen) { return v is string && v.size() <= maxLen; }
    function optStr(v) { return v == null || (v is string && v.size() <= 40); }


    // ให้แอปตรวจว่าติดตั้งกติการุ่นที่ต้องการแล้ว (อ่านเอกสารที่ไม่มีอยู่จริงได้ = กติการุ่นนี้)
    match /rulesVersion/{v} {
      allow get: if v == 'v2';
    }
```

```
    // ---------- โรงเรียน ----------
    match /school/info {
      allow get: if verified();
      // สร้างโรงเรียนได้ครั้งเดียว พร้อมกับบัญชีผู้ดูแลคนแรกใน batch เดียวกัน
      allow create: if verified() && request.resource.data.createdBy ==
request.auth.uid
        && isStr(request.resource.data.name, 120)
        && getAfter(memberPath()).data.role == 'ADMIN';
      allow update: if isAdmin() && request.resource.data.createdBy ==
resource.data.createdBy;
      allow delete: if false;
    }


    // ---------- สมาชิก ----------
    function memberShape(d) {
      return d.keys().hasOnly(['email', 'displayName', 'role', 'teacherId',
'studentId', 'childIds', 'active'])
        && isStr(d.email, 200) && isStr(d.displayName, 120)
        && d.role in ['ADMIN', 'TEACHER', 'STUDENT', 'PARENT']
        && optStr(d.teacherId) && optStr(d.studentId) && d.childIds is list &&
d.childIds.size() <= 20
        && d.active is bool;
    }
    function invitePath() { return
/databases/$(database)/documents/invites/$(myEmail()); }


    match /members/{uid} {
      allow get: if (signedIn() && request.auth.uid == uid) || isAdmin();
      allow list: if isAdmin();
      allow create: if verified() && request.auth.uid == uid &&
memberShape(request.resource.data)
        && request.resource.data.email == myEmail() &&
request.resource.data.active == true
        && (
          // ผู้ดูแลคนแรก: ยังไม่มีโรงเรียน และสร้างโรงเรียนใน batch เดียวกัน
          (request.resource.data.role == 'ADMIN'
            && !exists(/databases/$(database)/documents/school/info)
            && existsAfter(/databases/$(database)/documents/school/info)
            &&
getAfter(/databases/$(database)/documents/school/info).data.createdBy ==
request.auth.uid)
          // รับคำเชิญ: ค่าทุกช่องต้องตรงกับคำเชิญที่ผู้ดูแลสร้างให้อีเมลนี้
          || (exists(invitePath())
            && request.resource.data.role == dataAt(invitePath()).role
            && request.resource.data.displayName ==
dataAt(invitePath()).displayName
            && request.resource.data.teacherId == dataAt(invitePath()).teacherId
            && request.resource.data.studentId == dataAt(invitePath()).studentId
            && request.resource.data.childIds == dataAt(invitePath()).childIds)
        );
      // ผู้ดูแลแก้สมาชิกได้ แต่ห้ามลดสิทธิ์/ปิดบัญชีตนเอง และห้ามเปลี่ยนอีเมล
      allow update: if isAdmin() && memberShape(request.resource.data)
        && request.resource.data.email == resource.data.email
        && (uid != request.auth.uid || (request.resource.data.role == 'ADMIN' &&
request.resource.data.active == true));
      // ผู้ดูแลลบสมาชิกคนอื่นได้ / ทุกคนลบบัญชีสมาชิกของตนเองได้ (ข้อกำหนด Google Play)
```

```
      allow delete: if (isAdmin() && uid != request.auth.uid) || (signedIn() &&
request.auth.uid == uid);
    }


    match /invites/{email} {
      allow get: if (verified() && myEmail() == email) || isAdmin();
      allow list: if isAdmin();
      allow create, update: if isAdmin() && email == email.lower()
        && request.resource.data.keys().hasOnly(['email', 'displayName', 'role',
'teacherId', 'studentId', 'childIds', 'createdBy', 'createdAt'])
        && request.resource.data.email == email
        && request.resource.data.role in ['ADMIN', 'TEACHER', 'STUDENT', 'PARENT']
        && isStr(request.resource.data.displayName, 120)
        && optStr(request.resource.data.teacherId) &&
optStr(request.resource.data.studentId)
        && request.resource.data.childIds is list &&
request.resource.data.childIds.size() <= 20
        && request.resource.data.createdBy == request.auth.uid;
      // ผู้รับคำเชิญลบได้หลังรับ (ต้องมีบัญชีสมาชิกแล้วใน batch เดียวกัน)
      allow delete: if isAdmin() || (verified() && myEmail() == email &&
existsAfter(memberPath()));
    }


    // ---------- ข้อมูลหลัก ----------
    match /teachers/{id} {
      allow read: if isStaff();
      allow write: if isAdmin();
    }
    match /classrooms/{id} {
      allow read: if isMember();
      allow write: if isAdmin();
    }
    match /students/{id} {
      allow get: if isStaff() || (isMember() && (me().studentId == id || id in
me().childIds));
      allow list: if isStaff();
      allow write: if isAdmin();
    }
    match /subjects/{id} {
      allow read: if isMember();
      allow create, delete: if isAdmin();
      // ครูผู้สอนแก้ได้เฉพาะรายการประเมินของวิชาตน
      allow update: if isAdmin()
        || (teaches(id) &&
request.resource.data.diff(resource.data).affectedKeys().hasOnly(['assessments'])
            && request.resource.data.assessments is list &&
request.resource.data.assessments.size() <= 50);
    }
    match /settings/{name} {
      allow read: if isMember();
      allow write: if isAdmin();
    }


    // ---------- คะแนน ----------
    // เอกสารละ 1 นักเรียน x 1 วิชา (id = subjectId_studentId)
teacherId/classroomId/advisorId ต้องตรงกับข้อมูลปัจจุบัน
    function validScore(id, d) {
```

```
      return id == d.subjectId + '_' + d.studentId
        && d.keys().hasOnly(['subjectId', 'studentId', 'classroomId', 'teacherId',
'advisorId', 'values', 'updatedAt', 'updatedBy'])
        && d.values is map && d.values.size() <= 50
        && d.updatedBy == request.auth.uid && d.updatedAt == request.time
        && d.teacherId == subject(d.subjectId).teacherId
        && d.classroomId == subject(d.subjectId).classroomId
        && d.advisorId == classroom(d.classroomId).advisorId;
    }
    function readsScoreOf(d) {
      return isAdmin()
        || (isTeacher() && myTeacherId() != null && (d.teacherId == myTeacherId()
|| d.advisorId == myTeacherId()))
        || (isMember() && me().role == 'STUDENT' && d.studentId == me().studentId)
        || (isMember() && me().role == 'PARENT' && d.studentId in me().childIds);
    }
    match /scores/{id} {
      allow read: if readsScoreOf(resource.data);
      allow create, update: if canEditSubject(request.resource.data.subjectId) &&
validScore(id, request.resource.data)
        && (resource == null || (resource.data.subjectId ==
request.resource.data.subjectId && resource.data.studentId ==
request.resource.data.studentId));
      allow delete: if isAdmin();
    }


    // สรุปคะแนนรวมรายวิชา (ร้อยละของแต่ละคน) ใช้ทำแดชบอร์ด/รายงานรายห้องโดยไม่ต้องอ่านคะแนนทุกเอกสาร
    match /subjectStats/{subjectId} {
      allow read: if isAdmin()
        || (isTeacher() && myTeacherId() != null && (resource.data.teacherId ==
myTeacherId() || resource.data.advisorId == myTeacherId()));
      allow create, update: if canEditSubject(subjectId)
        && request.resource.data.keys().hasOnly(['subjectId', 'classroomId',
'teacherId', 'advisorId', 'percents', 'updatedAt'])
        && request.resource.data.subjectId == subjectId
        && request.resource.data.teacherId == subject(subjectId).teacherId
        && request.resource.data.classroomId == subject(subjectId).classroomId
        && (request.resource.data.classroomId == null ||
request.resource.data.advisorId ==
classroom(request.resource.data.classroomId).advisorId)
        && request.resource.data.percents is map;
      allow delete: if isAdmin();
    }


    // ประวัติแก้คะแนน: เพิ่มได้อย่างเดียว ผู้แก้ไขและเวลาต้องเป็นของจริง
    match /audit/{id} {
      allow read: if isAdmin() || (isTeacher() && myTeacherId() != null &&
resource.data.teacherId == myTeacherId());
      allow create: if canEditSubject(request.resource.data.subjectId)
        && request.resource.data.keys().hasOnly(['subjectId', 'teacherId',
'studentId', 'studentName', 'subjectName',
             'assessmentName', 'oldScore', 'newScore', 'by', 'byUid', 'at',
'reason'])
        && request.resource.data.byUid == request.auth.uid
        && request.resource.data.at == request.time
        && request.resource.data.teacherId ==
subject(request.resource.data.subjectId).teacherId
        && isStr(request.resource.data.reason, 500);
```

```
      allow update: if false;
      allow delete: if isAdmin();
    }
  }
}
```

ภาคผนวก ข — สำหรับผู้พัฒนาแอป: เปิดปุ่มเข้าสู่ระบบด้วย Google

ทำครั้งเดียวโดยผู้เผยแพร่แอป (ไม่ใช่โรงเรียน)

1. สร้างโปรเจกต์ใน Google Cloud Console หนึ่งโปรเจกต์ (โปรเจกต์กลางของผู้พัฒนา)
2. ตั้ง **OAuth consent screen** (External) ใส่ชื่อแอปและอีเมลติดต่อ
3. สร้าง **OAuth client ID ชนิด Android**: package `com.studentscore.app` + SHA-1 ของ **กุญแจ Play App Signing** (Play Console ⟶ Test and release ⟶ App integrity) และ SHA-1 ของกุญแจ debug สำหรับทดสอบ
4. สร้าง **OAuth client ID ชนิด Web application** — ค่านี้คือ Web client ID
5. ใส่ Web client ID ใน `app/build.gradle.kts` ที่ `GOOGLE_WEB_CLIENT_ID` แล้ว build ใหม่
6. แจ้ง Web client ID ให้โรงเรียนใช้ในขั้นที่ 6.3

> ห้ามเพิ่ม SHA-1 ของแอปใน Firebase ของโรงเรียน เพราะ Google ไม่ให้ใช้คู่ package + SHA-1 ซ้ำในหลายโปรเจกต์

ภาคผนวก ค — สำหรับผู้พัฒนา: ทดสอบกติกาสิทธิ์

```
cd firebase
npm install
npm test             (เทสต์กติกา 9 ชุดกับ Firestore Emulator)
npm run emulators    (เปิด emulator ไว้ แล้วรัน connectedDebugAndroidTest ทดสอบแอปทั้งเส้นทาง)
```